

```
In [1]: from google.colab import drive
        #drive.mount('/content/drive')
```

Mounted at /content/drive

```
In [1]: import pandas as pd
        #resign_df = pd.read_csv('/content/drive/MyDrive/2023 가을학기/resign.csv')
        resign_df = pd.read_csv('resign.csv')
        resign_df.head()
```

```
Out[1]:
```

	satisfaction	evaluation	project	workhour	years	accident	resign	promotion	good
0	0.38	0.53	2	157	3	0	1	0	0
1	0.80	0.86	5	262	6	0	1	0	1
2	0.11	0.88	7	272	4	0	1	0	1
3	0.72	0.87	5	223	5	0	1	0	1
4	0.37	0.52	2	159	3	0	1	0	0

- resign 데이터셋
 - 어느 회사의 인사관리에 관한 사례
 - 목표변수
 - resign : 각 직원의 퇴사여부 (0: 퇴사하지 않음, 1: 퇴사함)
 - 특성변수
 - satisfaction : 업무만족도
 - evaluation : 인사평점
 - project : 프로젝트 수행횟수
 - workhour : 월평균업무시간
 - years : 근속연수
 - accident : 사고여부
 - promotion : 최근 5년 이내 승진여부

```
In [3]: from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
resign_df_X = scaler.fit_transform(resign_df.iloc[:, [0, 1, 2, 3, 4, 5, 7]])
resign_df_Y = resign_df['resign']
```

```
In [5]: resign_df_X
```

```
Out[5]: array([[ -0.93649469, -1.08727529, -1.46286291, ..., -0.34123516,
        -0.41116529, -0.14741182],
        [ 0.75281433,  0.84070693,  0.97111292, ...,  1.71343614,
        -0.41116529, -0.14741182],
        [-2.02247906,  0.95755433,  2.59376348, ...,  0.34365527,
        -0.41116529, -0.14741182],
        ...,
        [-0.97671633, -1.08727529, -1.46286291, ..., -0.34123516,
        -0.41116529, -0.14741182],
        [-2.02247906,  1.42494396,  1.7824382 , ...,  0.34365527,
        -0.41116529, -0.14741182],
        [-0.97671633, -1.14569899, -1.46286291, ..., -0.34123516,
        -0.41116529, -0.14741182]])
```

```
In [6]: resign_df_Y
```

```
Out[6]: 0      1
1      1
2      1
3      1
4      1
..
14994   1
14995   1
14996   1
14997   1
14998   1
Name: resign, Length: 14999, dtype: int64
```

```
In [4]: from sklearn.model_selection import train_test_split
x_train, x_test, y_train, y_test = train_test_split(
    resign_df_X, resign_df_Y, test_size=0.2, random_state=0)
```

In [5]:

```
'''
코드는 TensorFlow를 사용하여 인공 신경망 모델을
만드는 과정을 보여줍니다.

1. `import tensorflow as tf`: TensorFlow 라이브러리를 임포트합니다.
2. `model = tf.keras.Sequential()`: Sequential 모델을 생성합니다.
   Sequential 모델은 레이어를 선형으로 쌓아 올린 신경망 구조를 갖습니다.
3. `model.add(tf.keras.layers.Dense(units=3, input_dim=7, activation='relu'))`:
   모델에 첫 번째 Dense 레이어를 추가합니다.
   이 레이어는 7개의 입력을 받아 3개의 뉴런으로 출력하는
   완전 연결 레이어입니다.
   `input_dim`은 입력의 차원을 의미합니다.
   활성화 함수로는 ReLU(Rectified Linear Unit)를 사용합니다.
   ReLU는 음수를 제거하고 양수를 그대로 반환하는 함수로,
   비선형성을 추가하여 모델이 복잡한 패턴을 학습할 수 있습니다.
4. `model.add(tf.keras.layers.Dense(units=1, activation='sigmoid'))`:
   모델에 두 번째 Dense 레이어를 추가합니다.
   이 레이어는 3개의 입력을 받아 하나의 뉴런으로 출력하는
   완전 연결 레이어입니다. 출력층이므로 `units`를 1로 설정합니다.
   활성화 함수로는 시그모이드(Sigmoid) 함수를 사용합니다.
   시그모이드 함수는 출력을 0과 1 사이로 제한하여
   이진 분류 문제에서 확률 값을 출력하는데 사용됩니다.

이렇게 구성된 모델은 입력층과 출력층 사이에 하나의 은닉층을 가지는
인공 신경망입니다. 입력층과 은닉층은 완전 연결된(Dense) 레이어로
구성되어 있고, 활성화 함수로는 ReLU가 사용됩니다.
출력층은 이진 분류를 위한 하나의 뉴런을 가지고 있고,
활성화 함수로는 시그모이드 함수가 사용됩니다.
'''

import tensorflow as tf
model = tf.keras.Sequential()
model.add(tf.keras.layers.Dense (units=3, input_dim=7,
                                activation='relu'))
model.add(tf.keras.layers.Dense (units=1,
                                activation='sigmoid'))
```

In [6]:

```
'''
`model.summary()` 함수는 TensorFlow의 Keras 모델에서 사용되는 메서드로,
모델의 구조와 파라미터 수 등을 요약하여 출력합니다.
'''
```

이를 통해 모델의 아키텍처를 쉽게 이해하고 디버깅하는 데 도움이 됩니다.

`model.summary()` 함수의 출력 내용은 다음과 같습니다:

1. **Layer (type):** 각 레이어의 이름과 유형입니다.
여기서는 'Dense' 레이어가 사용되었습니다.
 2. **Output Shape:** 각 레이어의 출력 형태입니다.
(배치 크기, 출력 차원)으로 표시됩니다.
 3. **Param #:** 각 레이어의 파라미터 수입니다.
이는 가중치(weight)와 편향(bias)을 합한 값입니다.
 4. **Total params:** 모델의 총 파라미터 수입니다.
모든 레이어의 파라미터 수를 합한 값입니다.
 5. **Trainable params:** 학습 가능한 파라미터 수입니다.
이는 모델이 역전파(backpropagation) 과정에서 업데이트되는
파라미터의 수를 의미합니다.
 6. **Non-trainable params:** 학습 불가능한 파라미터 수입니다.
이는 주로 Batch Normalization 레이어에서 사용되는데,
이러한 파라미터들은 학습 중에 업데이트되지 않습니다.
- ...

```
model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
=====		
dense (Dense)	(None, 3)	24
dense_1 (Dense)	(None, 1)	4
=====		
Total params: 28 (112.00 Byte)		
Trainable params: 28 (112.00 Byte)		
Non-trainable params: 0 (0.00 Byte)		

In [7]:

```
'''
`model.compile()` 함수는 TensorFlow의 Keras 모델에서 사용되는
메서드로, 모델을 컴파일하는 과정을 수행합니다.
모델을 컴파일하는 것은 학습 과정에서 사용할 손실 함수,
옵티마이저, 그리고 평가 지표(metrics)를 지정하는 것을 의미합니다.
```

특징:

- 모델을 컴파일하는 과정에서 손실 함수(loss function), 옵티마이저(optimizer), 그리고 평가 지표(metrics)를 설정합니다.
- 이러한 설정은 모델이 학습을 진행하는 동안 손실을 최소화하고 최적의 가중치를 찾는 데 도움을 줍니다.
- 모델을 컴파일한 후에는 `model.fit()` 메서드를 사용하여 모델을 학습시킬 수 있습니다.

사용법:

```
model.compile(optimizer='adam',
loss='binary_crossentropy', metrics=['accuracy'])
```

인자:

- `optimizer`: 최적화 알고리즘을 지정합니다.
SGD(확률적 경사 하강법), Adam, RMSprop 등의 옵티마이저를 사용할 수 있습니다.
- `loss`: 손실 함수를 지정합니다. 이진 분류일 경우 'binary_crossentropy', 다중 분류일 경우 'categorical_crossentropy' 등을 사용합니다.
- `metrics`: 모델을 평가할 지표를 지정합니다.
주로 'accuracy', 'precision', 'recall', 'f1-score' 등이 사용됩니다.

리턴값:

`None`을 반환합니다.

예시:

```
model.compile(optimizer='adam', loss='binary_crossentropy',
metrics=['accuracy'])
```

이 예시에서는 Adam 옵티마이저를 사용하고, 이진 분류 문제를 다루기 때문에 'binary_crossentropy' 손실 함수를 사용합니다. 또한 평가 지표로는 'accuracy'를 사용합니다. 이렇게 모델을 컴파일하면 학습 과정에서 지정된 손실 함수를 최소화하고 지정된 평가 지표를 기준으로 모델을 평가할 수 있습니다.

```
model.compile(optimizer='sgd',
loss='binary_crossentropy',
metrics=['accuracy'])
```

```
In [8]: '''
`model.fit()` 는 TensorFlow의 Keras 모델에서 사용되는 메서드로,
모델을 학습시키는 과정을 수행합니다.
이 함수를 호출하면 모델이 주어진 데이터에 맞춰
가중치를 업데이트하여 손실을 최소화하도록 학습됩니다.

특징:
- `model.fit()` 함수를 호출하여 모델을 학습시킬 수 있습니다.
  이때 모델은 주어진 데이터에 맞춰 가중치를 조정하고 최적화됩니다.
- 주어진 데이터를 사용하여 모델을 학습하는 과정은 주어진
  에포크(epoch) 수 만큼 반복됩니다.
  한 번의 에포크는 전체 데이터셋에 대한 한 번의 순회를 의미합니다.
- 학습 중간에는 학습 과정에서의 손실 및 정확도 등의
  지표를 확인할 수 있습니다.
- 학습이 완료되면 모델은 학습된 가중치를 사용하여
  새로운 데이터에 대한 예측을 수행할 수 있습니다.

사용법:
model.fit(X_train, y_train, epochs=10, batch_size=32,
          validation_data=(X_val, y_val))

인자:
- `X_train`, `y_train`: 학습에 사용될 입력 데이터와 레이블입니다.
- `epochs`: 전체 데이터셋에 대해 학습을 반복하는 횟수입니다.
- `batch_size`: 한 번의 학습 단계에서 사용할 샘플의 개수입니다.
  미니 배치 경사 하강법을 사용할 때 유용합니다.
- `validation_data`: 검증용 데이터의 입력과 레이블을 지정합니다.
  학습 중간에 이 데이터를 사용하여 모델의 성능을 평가할 수 있습니다.

리턴값:
`History` 객체를 반환합니다. 이 객체는 학습 과정에서의
손실과 정확도 등의 지표를 포함하고 있습니다.

예시:
history = model.fit(X_train, y_train, epochs=10, batch_size=32,
                    validation_data=(X_val, y_val))

이 코드는 입력 데이터 `X_train`과 레이블 `y_train`을 사용하여
모델을 학습합니다. 10번의 에포크 동안 학습을 반복하며,
각 학습 단계에서는 32개의 샘플을 사용합니다.
```

또한 검증용 데이터인 `X\_val`과 `y\_val`을 사용하여
 학습 중간에 모델의 성능을 평가합니다.
 학습 과정에서의 손실과 정확도 등은 `history` 객체에 저장됩니다.

'''

`History` 객체는 TensorFlow의 Keras 모델에서 사용되는 클래스로,
 모델의 학습 과정에서 발생한 손실과 정확도 등의 지표를
 저장하는 데 사용됩니다. 주로 `model.fit()` 메서드를 호출한
 결과로 반환되며, 학습 이력을 추적하고 시각화하는 데 유용합니다.

특징:

- `History` 객체는 학습 과정에서 발생한 손실,
 정확도 등의 지표를 저장합니다.
- 학습 과정 중에 발생한 모든 지표는 에포크마다 기록됩니다.
- `model.fit()` 메서드의 반환값으로 얻을 수 있으며,
 이를 통해 학습 이력을 확인하고 모델의 성능을 평가할 수 있습니다.

주요 속성:

- `history.history`: 학습 과정에서 발생한 지표를
 딕셔너리 형태로 저장합니다.
 각 지표는 에포크마다 리스트로 저장되어 있습니다.
 예를 들어, ``'loss'``, ``'accuracy'``, ``'val\_loss'``,
 ``'val\_accuracy'`` 등이 저장될 수 있습니다.

- 사용법:

```
history = model.fit(X_train, y_train, epochs=10,
                    validation_data=(X_val, y_val))
print(history.history['loss']) # 학습 손실
print(history.history['accuracy']) # 학습 정확도
print(history.history['val_loss']) # 검증 손실
print(history.history['val_accuracy']) # 검증 정확도
```

위의 예시에서는 `model.fit()` 메서드를 호출하여 모델을 학습하고,
 반환된 `history` 객체를 통해 학습 이력을 확인합니다.
 ``'loss'``, ``'accuracy'``, ``'val\_loss'``, ``'val\_accuracy'`` 등의
 키를 사용하여 손실과 정확도 지표를 확인할 수 있습니다.
 이러한 정보를 통해 학습 과정에서
 모델의 성능을 추적하고 시각화할 수 있습니다.

'''

```
result = model.fit(x_train, y_train, validation_split=0.2,  
                   epochs=40, verbose=1)
```



```
Epoch 1/40
300/300 [=====] - 2s 5ms/step - loss: 0.6017 - accuracy: 0.6976 - val_loss: 0.5476 - val_accuracy: 0.7538
Epoch 2/40
300/300 [=====] - 1s 2ms/step - loss: 0.5284 - accuracy: 0.7600 - val_loss: 0.5003 - val_accuracy: 0.7667
Epoch 3/40
300/300 [=====] - 1s 2ms/step - loss: 0.4878 - accuracy: 0.7799 - val_loss: 0.4599 - val_accuracy: 0.8029
Epoch 4/40
300/300 [=====] - 1s 2ms/step - loss: 0.4493 - accuracy: 0.8048 - val_loss: 0.4209 - val_accuracy: 0.8200
Epoch 5/40
300/300 [=====] - 1s 2ms/step - loss: 0.4109 - accuracy: 0.8066 - val_loss: 0.3831 - val_accuracy: 0.8129
Epoch 6/40
300/300 [=====] - 1s 2ms/step - loss: 0.3754 - accuracy: 0.8327 - val_loss: 0.3492 - val_accuracy: 0.8763
Epoch 7/40
300/300 [=====] - 1s 3ms/step - loss: 0.3445 - accuracy: 0.8799 - val_loss: 0.3209 - val_accuracy: 0.8975
Epoch 8/40
300/300 [=====] - 1s 2ms/step - loss: 0.3182 - accuracy: 0.8974 - val_loss: 0.2978 - val_accuracy: 0.9054
Epoch 9/40
300/300 [=====] - 1s 2ms/step - loss: 0.2962 - accuracy: 0.9026 - val_loss: 0.2780 - val_accuracy: 0.9108
Epoch 10/40
300/300 [=====] - 1s 2ms/step - loss: 0.2774 - accuracy: 0.9097 - val_loss: 0.2608 - val_accuracy: 0.9133
Epoch 11/40
300/300 [=====] - 1s 2ms/step - loss: 0.2609 - accuracy: 0.9166 - val_loss: 0.2455 - val_accuracy: 0.9204
Epoch 12/40
300/300 [=====] - 1s 4ms/step - loss: 0.2468 - accuracy: 0.9227 - val_loss: 0.2328 - val_accuracy: 0.9262
Epoch 13/40
300/300 [=====] - 1s 4ms/step - loss: 0.2352 - accuracy: 0.9265 - val_loss: 0.2225 - val_accuracy: 0.9325
Epoch 14/40
300/300 [=====] - 1s 4ms/step - loss: 0.2257 - accuracy: 0.9295 - val_loss: 0.2143 - val_accuracy: 0.9
```

```
367
Epoch 15/40
300/300 [=====] - 1s 3ms/step - loss: 0.2181 - accuracy: 0.9321 - val_loss: 0.2079 - val_accuracy: 0.9
388
Epoch 16/40
300/300 [=====] - 1s 2ms/step - loss: 0.2119 - accuracy: 0.9348 - val_loss: 0.2026 - val_accuracy: 0.9
425
Epoch 17/40
300/300 [=====] - 1s 2ms/step - loss: 0.2068 - accuracy: 0.9375 - val_loss: 0.1985 - val_accuracy: 0.9
429
Epoch 18/40
300/300 [=====] - 1s 2ms/step - loss: 0.2025 - accuracy: 0.9393 - val_loss: 0.1950 - val_accuracy: 0.9
450
Epoch 19/40
300/300 [=====] - 1s 3ms/step - loss: 0.1990 - accuracy: 0.9405 - val_loss: 0.1918 - val_accuracy: 0.9
454
Epoch 20/40
300/300 [=====] - 1s 2ms/step - loss: 0.1960 - accuracy: 0.9411 - val_loss: 0.1891 - val_accuracy: 0.9
471
Epoch 21/40
300/300 [=====] - 1s 2ms/step - loss: 0.1934 - accuracy: 0.9417 - val_loss: 0.1868 - val_accuracy: 0.9
488
Epoch 22/40
300/300 [=====] - 1s 2ms/step - loss: 0.1912 - accuracy: 0.9432 - val_loss: 0.1848 - val_accuracy: 0.9
483
Epoch 23/40
300/300 [=====] - 1s 2ms/step - loss: 0.1893 - accuracy: 0.9433 - val_loss: 0.1830 - val_accuracy: 0.9
496
Epoch 24/40
300/300 [=====] - 1s 2ms/step - loss: 0.1876 - accuracy: 0.9438 - val_loss: 0.1815 - val_accuracy: 0.9
496
Epoch 25/40
300/300 [=====] - 1s 2ms/step - loss: 0.1860 - accuracy: 0.9444 - val_loss: 0.1798 - val_accuracy: 0.9
500
Epoch 26/40
300/300 [=====] - 1s 2ms/step - loss: 0.1847 - accuracy: 0.9446 - val_loss: 0.1785 - val_accuracy: 0.9
529
Epoch 27/40
300/300 [=====] - 1s 2ms/step - loss: 0.1835 - accuracy: 0.9451 - val_loss: 0.1770 - val_accuracy: 0.9
533
Epoch 28/40
```

```

300/300 [=====] - 1s 2ms/step - loss: 0.1824 - accuracy: 0.9462 - val_loss: 0.1758 - val_accuracy: 0.9
533
Epoch 29/40
300/300 [=====] - 1s 2ms/step - loss: 0.1814 - accuracy: 0.9457 - val_loss: 0.1747 - val_accuracy: 0.9
533
Epoch 30/40
300/300 [=====] - 1s 3ms/step - loss: 0.1805 - accuracy: 0.9469 - val_loss: 0.1738 - val_accuracy: 0.9
538
Epoch 31/40
300/300 [=====] - 1s 3ms/step - loss: 0.1797 - accuracy: 0.9472 - val_loss: 0.1729 - val_accuracy: 0.9
542
Epoch 32/40
300/300 [=====] - 1s 3ms/step - loss: 0.1789 - accuracy: 0.9474 - val_loss: 0.1720 - val_accuracy: 0.9
550
Epoch 33/40
300/300 [=====] - 1s 3ms/step - loss: 0.1783 - accuracy: 0.9479 - val_loss: 0.1712 - val_accuracy: 0.9
558
Epoch 34/40
300/300 [=====] - 1s 3ms/step - loss: 0.1776 - accuracy: 0.9476 - val_loss: 0.1703 - val_accuracy: 0.9
558
Epoch 35/40
300/300 [=====] - 1s 2ms/step - loss: 0.1770 - accuracy: 0.9478 - val_loss: 0.1696 - val_accuracy: 0.9
558
Epoch 36/40
300/300 [=====] - 1s 2ms/step - loss: 0.1764 - accuracy: 0.9477 - val_loss: 0.1688 - val_accuracy: 0.9
558
Epoch 37/40
300/300 [=====] - 1s 2ms/step - loss: 0.1759 - accuracy: 0.9484 - val_loss: 0.1681 - val_accuracy: 0.9
550
Epoch 38/40
300/300 [=====] - 1s 2ms/step - loss: 0.1754 - accuracy: 0.9481 - val_loss: 0.1675 - val_accuracy: 0.9
546
Epoch 39/40
300/300 [=====] - 1s 2ms/step - loss: 0.1749 - accuracy: 0.9479 - val_loss: 0.1672 - val_accuracy: 0.9
554
Epoch 40/40
300/300 [=====] - 1s 2ms/step - loss: 0.1744 - accuracy: 0.9486 - val_loss: 0.1663 - val_accuracy: 0.9
546

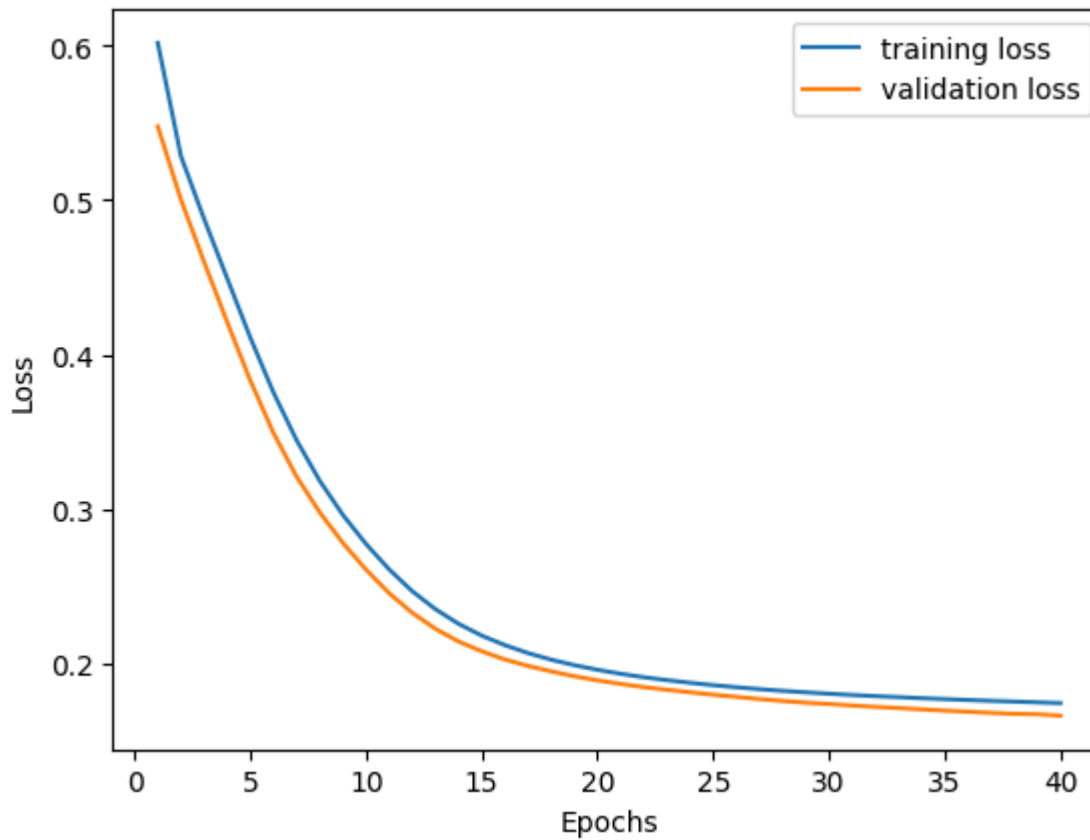
```

```

In [9]: import matplotlib.pyplot as plt
import numpy as np

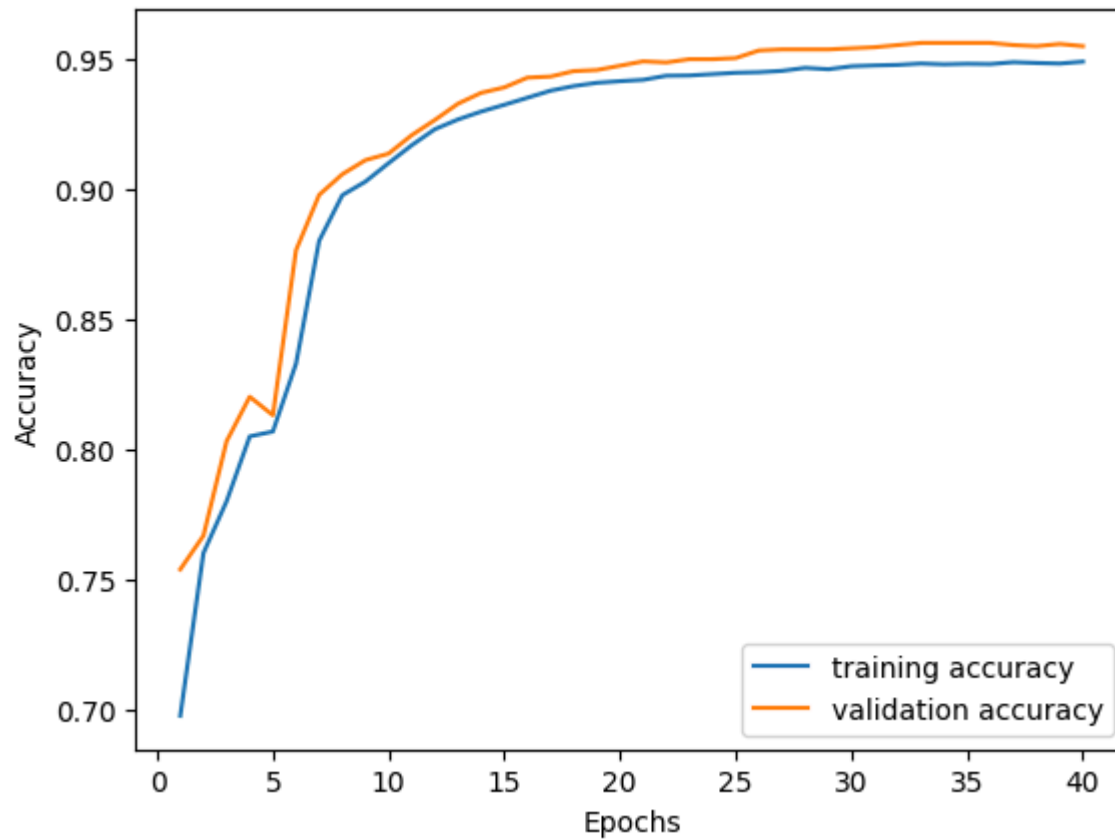
```

```
epochs=np.arange(1, 40+1)
plt.plot(epochs, result.history['loss'], label='training loss')
plt.plot(epochs, result.history['val_loss'], label='validation loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()
plt.show()
```



```
In [10]: epochs=np.arange(1, 40+1)
plt.plot(epochs, result.history['accuracy'],
         label='training accuracy')
plt.plot(epochs, result.history['val_accuracy'],
         label='validation accuracy')
plt.xlabel('Epochs')
```

```
plt.ylabel('Accuracy')  
plt.legend()  
plt.show()
```



```
In [11]: '''  
`model.evaluate()` 함수는 TensorFlow의 Keras 모델에서  
사용되는 메서드로, 학습된 모델의 성능을 평가하기 위해  
주어진 데이터에 대해 손실과 지정된 메트릭을 계산합니다.  
  
특징:  
- `model.evaluate()` 함수를 사용하여 모델의 성능을  
  평가할 수 있습니다. 이때 주어진 데이터에 대한 손실과  
  지정된 메트릭 값을 계산합니다.  
- 일반적으로 모델이 학습된 후에 검증 데이터셋이나 테스트
```

데이터셋에 대해 모델의 성능을 평가할 때 사용됩니다.

- `model.evaluate()` 함수는 모델의 손실과 지정된 메트릭 값을 반환합니다.

사용법:

```
loss, accuracy = model.evaluate(X_test, y_test)
```

인자:

- `X_test`, `y_test`: 평가에 사용할 입력 데이터와 레이블입니다.

리턴값:

- `loss`: 평가 데이터에 대한 손실 값입니다.
- `accuracy` 또는 지정된 메트릭: 평가 데이터에 대한 지정된 메트릭의 값입니다.

사용자가 지정한 메트릭에 따라 다를 수 있습니다.

예시:

```
loss, accuracy = model.evaluate(X_test, y_test)
print("Test Loss:", loss)
print("Test Accuracy:", accuracy)
```

위의 예시에서는 `model.evaluate()` 함수를 사용하여 평가 데이터셋인 `X_test`와 `y_test`를 이용하여 모델을 평가하고, 반환된 손실과 정확도 값을 출력합니다. 이를 통해 모델이 얼마나 좋은 성능을 보이는지를 확인할 수 있습니다.

```
'''
model.evaluate(x_test, y_test)
```

```
94/94 [=====] - 0s 2ms/step - loss: 0.1681 - accuracy: 0.9503
```

```
Out[11]: [0.16808192431926727, 0.9503333568572998]
```

```
In [12]: '''
x_test[0, ]: 이 부분은 테스트 데이터셋(x_test) 중
첫 번째 샘플을 선택하는 것을 나타냅니다.
배열 인덱싱을 사용하여 첫 번째 행을 선택하고,
각 특성 값들을 나타냅니다.
- satisfaction : 업무만족도
- evaluation : 인사평점
```

```

- project : 프로젝트 수행횟수
- workhour : 월평균업무시간
- years : 근속연수
- accident : 사고여부
- promotion : 최근 5년 이내 승진여부
이 부분은 모델에 입력으로 제공되는 데이터 중
첫 번째 샘플을 선택하는 것을 의미합니다.
'''

x_test[0, ]

```

```

Out[12]: array([-0.69516483, -0.85358047, -1.46286291, -1.20241514, -0.34123516,
               -0.41116529, -0.14741182])

```

```

In [13]: '''
코드는 주어진 입력 데이터 `x_new`를 사용하여 학습된
모델을 통해 예측을 수행하는 과정을 나타냅니다.
1. `x_new = [[-0.7, -0.9, -1.3, -0.9, 0.3, 0.4, -0.02]]`:
   이 부분은 새로운 입력 데이터 `x_new`를 정의하는 것입니다.
   입력 데이터는 모델에 입력되는 특성들의 값을 포함하는
   2차원 배열입니다. 각 행은 하나의 샘플을 나타내며,
   각 열은 해당 샘플의 특성 값을 나타냅니다.
2. `y_pred = model.predict(x_new)` :
   이 부분은 학습된 모델을 사용하여 주어진 입력 데이터
   `x_new`에 대해 예측을 수행하는 것을 의미합니다.
   `model.predict()` 함수는 입력 데이터를 사용하여
   모델의 출력을 예측합니다.
   여기서 예측된 출력은 주어진 입력 데이터에 대한
   모델의 예측값을 나타냅니다.
따라서 `y_pred`는 주어진 입력 데이터 `x_new`에 대한
모델의 예측값을 포함하는 배열이 됩니다.
이 예측값은 모델이 입력 데이터에 대해
어떤 출력을 예상하는지를 나타냅니다.
'''

x_new = [[-0.7, -0.9, -1.3, -0.9, 0.3, 0.4, -0.02]]
y_pred = model.predict(x_new)
y_pred

```

```

1/1 [=====] - 0s 96ms/step

```

```
Out[13]: array([[0.48450288]], dtype=float32)
```